

HelixMemory

HelixMemory

AI агенттері үшін бір жады миы — төрт жоғары деңгейлі қозғалтқыштың бірігуі.

Қысқаша мазмұны

HelixMemory — бұл AI қолданбалары үшін біртұтас когнитивтік жады қозғалтқышы ретінде әрекет ететін Go SDK. Ол төрт жетекші жады жүйесін (Mem0, Cognee, Letta, Graphiti) біріктіріп, оларды параллель іздейді және нәтижелерді біріктіреді. Бұл AI қолданбаларына төрт бөлек емес, бір тұрақты, қайталанбаған, қайта рейтингтелген жады қабатын береді.

Қысқаша сипаттама

HelixMemory — бұл AI қолданбалары үшін Mem0, Cognee, Letta және Graphiti жүйелерін біртұтас когнитивтік жады қозғалтқышына біріктіретін Go SDK. Ол жазуларды ақылды түрде бағыттайды, әр бекенді параллель іздейді және нәтижелерді үш кезеңді жинау-қайталау-қайта рейтингтеу конвейері арқылы біріктіреді.

Кеңейтілген сипаттама

HelixMemory — бұл AI қолданбалары үшін біртұтас когнитивтік жады қозғалтқышы, ол Go SDK ретінде (`digital.vasic.helixmemory` модулі, Go 1.25+) жеткізіледі. Оның негізгі болжамы — бір ғана жады жобасы әр нәрседе ең жақсы бола алмайды дегенге негізделген. Сондықтан жадыны нөлден қайта жасаудың орнына және бір жобаның соқыр жеріне ие болудың орнына, ол төрт жоғары деңгейлі жүйені оркестрлейді және әрқайсысына өз күшін көрсетуге мүмкіндік береді: Mem0 — динамикалық фактілерді бөліп алу және таңдауды басқару үшін, Cognee — ECL конвейерлері арқылы

құрылған семантикалық білім графикалары үшін, Letta — өңделетін жады блоктары мен ұйқы кезіндегі есептеулері бар күйлік агенттік орындау ортасы үшін, ал Graphiti — уақыт бойынша фактілердің өзгеруін талдауға мүмкіндік беретін битемпоралды білім графикасы үшін.

Біріктіру қозғалтқышы осы төрт тәуелсіз қоймаларды бір миға айналдырады. Жазу жолында әрбір келіп түскен жады мазмұны бойынша жіктеліп, оны сақтауға ең қолайлы бекендке бағытталады. Оқу жолында сұраныс барлық бекендтерге параллель түрде жіберіліп, шикі нәтижелер үш кезеңді біріктіру конвейеріне — жинау, қайталау және көлденең көздерді қайта рейтингтеу — түседі. Нәтижесінде шақырушыға төрт шулы, қайталанған нәтижелер жиынтығы емес, тек бір таза, рейтингтелген жауап көрсетіледі. Әр бекендті сенімділік қорғаушылары қоршап тұрады: бір қозғалтқыш істен шыққан кезде оның қорғаушысы іске қосылып, қалған бекендтер жалғастыра береді, бұл жады қабатын түгелдей құлатудың орнына. Қозғалтқыш MemoryStore интерфейсін тікелей ауыстыруға мүмкіндік беретін түрде жүзеге асырғандықтан, ол қарапайым жады провайдерінің орнына тікелей қойылады — шақырушыны қайта құрудың қажеті жоқ. Ал Prometheus метрикалары бағыттау мен біріктірудің ішкі процестерін толық бақылауға мүмкіндік береді.

HelixMemory HelixAgent үшін жады қабаты ретінде құрылды, бұл кеңірек Helix AI ансамблінің бөлігі. Ол отбасының алдауға қарсы сынақ әдістемесін жады саласына әкелді: процесс ішіндегі сынақ жүргізушісі нақты өндірістік код жолдарын — бағыттау, біріктіру, аудармашы, сенімділік қорғаушысын — тексереді, ал жұпты өзгерту ораушысы шарттарды әдейі бұзады, бұл логика бұзылған кезде сынақтардың шынымен сәтсіз болатынын дәлелдеу үшін. Сондықтан жасыл жолақ бірдеңе білдіреді.

Неліктен құрылды

Мазмұны

AI агенттері ұзақ мерзімді, жоғары сапалы жадыны қажет етеді, бірақ экосистема бөлінген — әрбір жады жобасы (Mem0, Cognee, Letta, Graphiti) бір нәрседе мықты, ал басқаларында әлсіз. HelixMemory HelixAgent-ге олардың күштерін біріктіретін, бірақ ешқайсысына тұйықталуды талап етпейтін біртұтас жады бетін беру үшін жасалды.

Неге бұл ойынды өзгертетін шешім

Мәжбүрлі таңдауды жойды. Әдетте бір орын үшін бәсекелес төрт жады жүйесі бір интерфейс арқылы қосымша серверлерге айналады — осылайша қосымша динамикалық факттарды бөліп алуды, семантикалық білім графтарын, агент жадысының күйін және битемпоралды ойлауды *бір мезгілде* алады, ал дедупликация мен көздерді қайта рейтингтеу автоматты түрде өңделеді. Бұрын мүмкін болмаған нәрсе — «қандай жады қозғалтқышын қабылдаймыз?» деген жалған дилемманы қарастыру: HelixMemory сізге бір ғана MemoryStore арқылы олардың барлық артықшылықтарын бірден алуға мүмкіндік береді, бірақ ешқандай қозғалтқыштың соқыр нүктесін мұраға алмай, тұйықталуға міндеттемейді.

Қандай жаңалықтар бар

- Көп серверлік **фузия** (жинау → дедупликация → көздерді қайта рейтингтеу), ол бір рейтингтелген нәтижелер жиынтығын қайтарады, бір ғана қоймаға байланыстырудың орнына.
- **Ақылды жазу маршруттауы**, ол әрбір жадыны мазмұны бойынша жіктейді және оны ұстауға ең қолайлы қозғалтқышқа жібереді, осылайша дұрыс деректер дұрыс қоймаға түседі.
- **Жұмсақ деградация** серверлер бойынша схемалық бұзушылар арқылы — істен шыққан қозғалтқыш оқшауланып, өлімге әкелмейді, ал қалғандары қызмет көрсетуді жалғастырады.
- **Ұйқы уақытындағы есептеулерді** біріктіру (Letta арқылы), ол жадыны бос уақытта қайта өңдейді, тек сұрау уақытында ғана емес.
- **Алаяқтықтың алдын алу тексеруі**: нақты өндірістік кодтағы сынақ жүргізушісі, оны өзгерту орағымен жұптастырылған, ол инвариант бұзылғанда сәтсіз болуы керек — бұл сынақ қақпасының нақты тексеру екенін, тавтология емес екенін дәлелдейді.

Ең үлкен техникалық қиындықтар және оларды қалай шештік

- **Төрт әр түрлі серверді біртұтас нәтижелер жиынына біріктіру** — әрбір қозғалтқыш жадыны өз пішінінде қайтарады, ал оларды қарапайым біріктіру дубликаттар мен салыстыруға келмейтін рейтингтерге әкеледі. Бұл мәселе типтелген фузия қозғалтқышымен шешілді, ол көздер бойынша жинайды, қайталауларды жою арқылы дедупликация жасайды және барлығын ортақ негізде қайта

рейтингтейді, ал біріктірілген санақ инварианты сынақтарда расталады, осылайша біріктіру нәтижелерді жасырын жоғалтпайды немесе екі еселеп санамайды.

- **Сервер істен шыққан кезде жұмыс істей беру** — бір қол жетпейтін жады қозғалтқышы бүкіл қабатты тоқтата алмауы керек. Бұл серверлер бойынша схемалық бұзушылармен шешілді, олар жабық → ашық (ісінгеннен кейін) → жартылай ашық (уақыт өткеннен кейін) күйлер машинасы арқылы жұмыс істейді, істен шыққан серверді оқшаулайды және дені сау серверлерден қызмет көрсетуді жалғастырады, олар қалпына келгенше.
- **Жады логикасының шынымен жұмыс істейтінін, тек құрастырылатынын емес, дәлелдеу** — жасыл сынақтар жиыны сынақтар сәтсіз бола алмаса, мағынасыз. Бұл нақты өндірістік кодты (маршруттау, фузия, аудармашы, схемалық бұзушы) жүргізетін процесс ішіндегі сынақ жүргізушісімен және инварианттарды бұзатын жұптастырылған өзгерту орағымен шешілді, ол сынақтардың қызыл түсуін талап етеді, осылайша қақпа нақты тексеру екенін, тавтология емес екенін дәлелдейді.

Мазмұны

Технологиялық стегі

- **Go (1.25+)** — жалғыз SDK және оның орындау ортасы; төрт бекенд бойынша параллельді оқу таратуы бір уақытша мәселе болып табылатындықтан тандалды, ал Go-тың горутиндері оны арзан етеді, ал оның интерфейс типтері бүкіл жүйеге бір таза тігіс (MemoryStore) береді, оны шақырушылар пайдалана алады.
- **Mem0** — динамикалық факттарды шығару және басымдықтарды басқару бекенді; пайдаланушының шын мәнінде неге басымдық беретінін / қандай факттар ашылғанын анықтау үшін қолданылады.
- **Cognee** — ECL құбырлары негізінде жасалған семантикалық білімдер графі бекенді; құрылымдалған, байланысты білімдерді сақтау үшін қолданылады, ал жай факттарды емес.
- **Letta** — өңделетін жады блоктары мен ұйқы уақытындағы есептеулері бар күйлі агент-орындау бекенді; жады агент күйі ретінде сақталуы керек және бос уақытта біріктірілуі тиіс жерлерде қолданылады.
- **Graphiti** — екі уақыттық білімдер графі бекенді; факттар мен байланыстардың уақыт бойынша қалай өзгергенін, тек қазіргі мәндерін ғана емес, түсіну үшін қолданылады.
- **PostgreSQL + Neo4j + Redis** — бекендтер жұмыс істейтін нақты дерекқорлар, make infra-start арқылы нақты интеграциялық тестілеу үшін орнатылған, сондықтан

топтама тірі инфрақұрылымды, ал жалған объектілерді емес, тексереді.

- **Prometheus** — метрикалар мен бақылау функциялары фьюжн құбыры арқылы қосылған, сондықтан маршруттау мен фьюжн әрекеттері өндірісте өлшенетін болады, қара қорап емес.
- **i18n аудармашы тігісі** — (helixmemory_) атау кеңістігіндегі жолдық бет, яғни болашақта пайдаланушыға көрсетілетін қабатты жергілікті ету үшін ядроны қайта жабдықтаудың қажеті жоқ.

Жағдай және ашықтық ескертпелері

- **Жағдайы: бета.** Жұмыс істейтін SDK; HelixAgent үшін жады қабаты ретінде жасалған.
- **Лицензиясы: анықталмаған.** GitHub API арқылы LICENSE табылмады — ТЕКСЕРІЛМЕГЕН / жарияланбаған.
- “HelixMemory” атауы memory репозиторийіне сәйкес келеді. README-да келтірілген дәлдік көрсеткіштері жоғарыдағы жеткізушілердің мәліметтері, HelixMemory өлшемдері емес, сондықтан мұнда келтірілмеген.

Басымдық деңгейі: Helix-негізгі.